



电子科技大学
格拉斯哥海南学院
Glasgow College Hainan, UESTC

UESTC 4014: Real-Time Computing Systems and Architecture

Fall Semester 2026-2027

Lab 2 - Week 2

Report Due Date: Wednesday, 30th Sept 2026, 23:59 CST

Student's Name	Zhanyu Zhang
Student's UoG ID	2959456
Student's UESTC ID	2023300901013
Date	18 September 2026

Abstract

This laboratory implements UART, I2C, and SPI communication together with pulse-width modulation. A UART-driven calculator receives three two-digit integers and reports either their sum or product. The STM32 then operates as an I2C master to control an Arduino LED interval and read either a two-byte analog sample or a four-byte character sequence. The same analog sample is acquired through SPI and mapped first to an LED duty cycle and then to a servo command pulse. The four tasks combine interrupt-driven terminal input, structured peripheral transactions, byte reconstruction, and timer compare control in one modular application.

1 Introduction

Embedded systems commonly combine several communication interfaces because each protocol serves a different purpose. UART provides a direct text interface, I2C connects addressed devices on a two-wire bus, and SPI offers synchronous full-duplex transfers controlled by chip select. PWM converts numerical control values into timed output waveforms suitable for LED brightness and servo position control.

The application is organised as a UART menu with a state machine. Received characters are assembled into complete lines by the USART2 interrupt callback, while the main loop parses commands and executes the selected task. I2C1 and SPI1 communicate with the supplied Arduino slave programs. TIM2 and TIM21 generate the two PWM outputs.

2 Task 1: UART Calculator

USART2 is configured for asynchronous communication at 115200 bit/s with eight data bits, no parity, and one stop bit. Reception is interrupt driven one byte at a time. When a carriage return or line feed is received, the completed line is processed by the application state machine.

The first line contains three space-separated two-digit integers. The next input selects addition or multiplication. Listing 1 shows the parsing and arithmetic module. A 32-bit result is used so that the largest permitted product, $99^3 = 970299$, is represented directly.

Listing 1: UART calculator parsing and arithmetic.

```
1 #include "task1_uart_calc.h"
2
3 #include <stdio.h>
4
5 bool Task1_ParseNumbers(const char *text, uint8_t values[3])
6 {
7     unsigned int a;
8     unsigned int b;
9     unsigned int c;
10
11     /* Convert the three space-separated fields received from the terminal. */
12     if (sscanf(text, "%u %u %u", &a, &b, &c) != 3) {
13         return false;
14     }
15
16     /* A two-digit decimal integer is in the inclusive range 10 to 99. */
17     if ((a < 10U) || (a > 99U)
18         || (b < 10U) || (b > 99U)
19         || (c < 10U) || (c > 99U)) {
20         return false;
21     }
22
23     values[0] = (uint8_t)a;
24     values[1] = (uint8_t)b;
25     values[2] = (uint8_t)c;
26     return true;
27 }
28
29 bool Task1_Calculate(const uint8_t values[3], char operation, uint32_t *result)
30 {
31     /* Accept the menu letter and the familiar arithmetic symbol. */
```

```

32 if ((operation == 'A') || (operation == 'a') || (operation == '+')) {
33     *result = (uint32_t)values[0] + values[1] + values[2];
34     return true;
35 }
36
37 if ((operation == 'M') || (operation == 'm') || (operation == '*')) {
38     *result = (uint32_t)values[0] * values[1] * values[2];
39     return true;
40 }
41
42 return false;
43 }

```

For the test data 12, 34, and 56, the two requested operations are

$$12 + 34 + 56 = 102, \quad 12 \times 34 \times 56 = 22848.$$

Figure 1 identifies the two serial-monitor result captures and the exact test sequence used for each case.

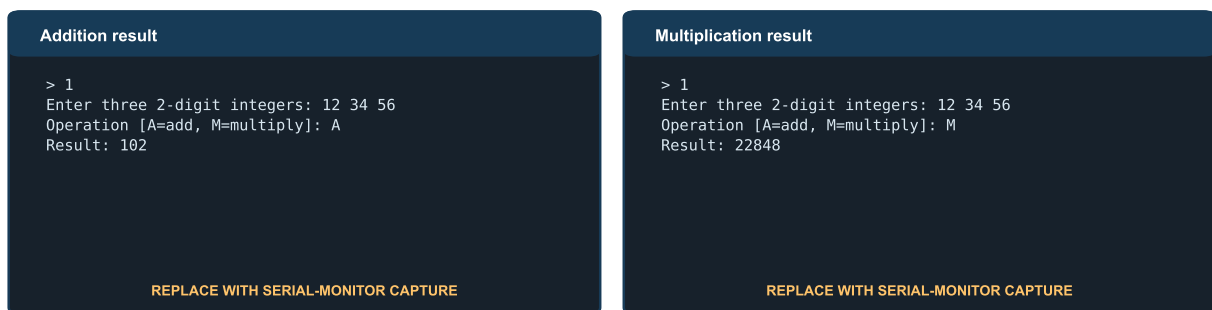


Figure 1: Task 1 serial-monitor result placeholders for addition and multiplication.

3 Task 2: I2C Command and Data Selection

The STM32 is the I2C1 master and the Arduino is the slave at seven-bit address 0x55. The first UART selection is converted to one of the commands 0x80–0x83, which selects the Arduino LED interval. The second selection chooses register 0x00 or 0x01.

Register 0x00 returns a two-byte analog sample in little-endian order. If the received bytes are b_0 and b_1 , the value is reconstructed as

$$A = b_0 + (b_1 \ll 8).$$

Register 0x01 returns the four ASCII bytes RTCA. Listing 2 implements the command write, register selection, read transaction, and data reconstruction.

Listing 2: I2C command and data exchange.

```

1  #include "task2_i2c.h"
2
3  #define ARDUINO_I2C_ADDRESS (0x55U << 1U) /* HAL uses the shifted address. */
4  #define I2C_TIMEOUT_MS 100U
5
6  HAL_StatusTypeDef Task2_Exchange(I2C_HandleTypeDef *hi2c,
7                                  uint8_t speed_selection,
8                                  uint8_t data_selection,
9                                  Task2_Result *result)
10 {
11     uint8_t command = (uint8_t)(0x80U + speed_selection);
12     uint8_t received[4] = {0U};
13     uint16_t length = (data_selection == 0U) ? 2U : 4U;
14     HAL_StatusTypeDef status;
15
16     /* First write selects the Arduino LED interval (commands 0x80--0x83). */

```

```

17 status = HAL_I2C_Master_Transmit(hi2c, ARDUINO_I2C_ADDRESS,
18                                 &command, 1U, I2C_TIMEOUT_MS);
19 if (status != HAL_OK) {
20     return status;
21 }
22
23 /* Second write selects register 0x00 or 0x01 for the following read. */
24 status = HAL_I2C_Master_Transmit(hi2c, ARDUINO_I2C_ADDRESS,
25                                 &data_selection, 1U, I2C_TIMEOUT_MS);
26 if (status != HAL_OK) {
27     return status;
28 }
29
30 status = HAL_I2C_Master_Receive(hi2c, ARDUINO_I2C_ADDRESS,
31                                received, length, I2C_TIMEOUT_MS);
32 if (status != HAL_OK) {
33     return status;
34 }
35
36 if (data_selection == 0U) {
37     /* The Arduino transmits the 10-bit sample in little-endian order. */
38     result->analog_value = (uint16_t)received[0]
39                          | ((uint16_t)received[1] << 8U);
40 } else {
41     /* Register 0x01 contains four ASCII bytes; add a C terminator locally. */
42     result->text[0] = (char)received[0];
43     result->text[1] = (char)received[1];
44     result->text[2] = (char)received[2];
45     result->text[3] = (char)received[3];
46     result->text[4] = '\0';
47 }
48
49 return HAL_OK;
50 }

```

I2C transaction sequence

Stage	Transferred value	Purpose
Write 1	0x80–0x83	Select LED interval
Write 2	0x00 or 0x01	Select slave data
Read	2 or 4 bytes	Acquire analog sample or text

Figure 2 provides the two required serial-monitor capture locations. The example uses speed selection 2, which sends 0x82; the left case reads register 0x00, and the right case reads register 0x01.

I2C register 0x00

```

> 2
Enter speed 0..3 and data 0..1: 2 0
Command 0x82, register 0x00,
analog value: 673

```

REPLACE WITH 2-BYTE ANALOG CAPTURE

I2C register 0x01

```

> 2
Enter speed 0..3 and data 0..1: 2 1
Command 0x82, register 0x01,
text: RTCA

```

REPLACE WITH 4-BYTE DATA CAPTURE

Figure 2: Task 2 serial-monitor result placeholders for the two-byte analog value and four-byte text.

4 Task 3: SPI Analog Input to LED PWM

SPI1 operates as a mode-0, MSB-first master with a prescaler of 128. The 32 MHz peripheral clock therefore produces

$$f_{\text{SCK}} = \frac{32 \text{ MHz}}{128} = 250 \text{ kHz}.$$

The Arduino exposes the lower and upper ADC bytes through commands 0x00 and 0x01. Each register read sends its command and then a dummy byte to generate the clocks for the reply. The two bytes are combined as a 16-bit value.

TIM2 generates the LED waveform. With PSC = 31 and ARR = 999,

$$f_{\text{PWM}} = \frac{32\,000\,000}{(31 + 1)(999 + 1)} = 1 \text{ kHz}.$$

For a 10-bit Arduino sample $A \in [0, 1023]$, the compare value is

$$CCR_{\text{LED}} = \left\lfloor \frac{1000A}{1023} \right\rfloor, \quad D = 100 \frac{CCR_{\text{LED}}}{1000} \%.$$

Thus $A = 0$ produces 0%, $A = 512$ produces approximately 50%, and $A = 1023$ produces 100% duty cycle. The SPI transaction and LED mapping are shown together in Listing 3.

5 Task 4: SPI Analog Input to Servo PWM

TIM21 generates the servo control waveform. The same prescaler gives a 1 MHz counter, so one timer count represents 1 μs . With ARR = 19999, the PWM period is

$$T_{\text{servo}} = \frac{(31 + 1)(19999 + 1)}{32\,000\,000} = 20 \text{ ms}, \quad f_{\text{servo}} = 50 \text{ Hz}.$$

The analog input is mapped linearly to a pulse width from 1000 μs to 2000 μs :

$$CCR_{\text{servo}} = 1000 + \left\lfloor \frac{1000A}{1023} \right\rfloor.$$

Under a nominal 0°–180° servo mapping, the corresponding command angle is

$$\theta = \left\lfloor \frac{180A}{1023} \right\rfloor.$$

The end points therefore command 1000 μs at 0° and 2000 μs at 180°, while the midpoint is approximately 1500 μs at 90°.

Listing 3: SPI analog acquisition and PWM mappings for Tasks 3 and 4.

```
1 #include "task34_spi_pwm.h"
2
3 #include "main.h"
4
5 #define ARDUINO_ADC_MAX 1023U
6 #define LED_PWM_COUNTS 1000U
7 #define SERVO_MIN_PULSE_US 1000U
8 #define SERVO_PULSE_RANGE_US 1000U
9 #define SPI_TIMEOUT_MS 100U
10
11 static HAL_StatusTypeDef ReadRegister(SPI_HandleTypeDef *hspi,
12                                     uint8_t address,
13                                     uint8_t *value)
14 {
15     uint8_t discard;
16     uint8_t dummy = 0xFFU;
17     HAL_StatusTypeDef status;
18
19     /* The address byte loads the selected value into the Arduino SPI register. */
```

```

20 HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_RESET);
21 status = HAL_SPI_TransmitReceive(hspi, &address, &discard, 1U,
22                                 SPI_TIMEOUT_MS);
23 if (status == HAL_OK) {
24     HAL_Delay(1U);
25     /* A dummy byte supplies the clocks that return the selected data byte. */
26     status = HAL_SPI_TransmitReceive(hspi, &dummy, value, 1U,
27                                     SPI_TIMEOUT_MS);
28 }
29 HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_SET);
30 return status;
31 }
32
33 HAL_StatusTypeDef Task34_ReadAnalog(SPI_HandleTypeDef *hspi,
34                                     uint16_t *analog_value)
35 {
36     uint8_t lower;
37     uint8_t upper;
38     HAL_StatusTypeDef status;
39
40     /* The supplied slave exposes the ADC sample as lower and upper bytes. */
41     status = ReadRegister(hspi, 0x00U, &lower);
42     if (status != HAL_OK) {
43         return status;
44     }
45
46     status = ReadRegister(hspi, 0x01U, &upper);
47     if (status == HAL_OK) {
48         *analog_value = (uint16_t)lower | ((uint16_t)upper << 8U);
49     }
50     return status;
51 }
52
53 uint32_t Task3_SetLedDuty(TIM_HandleTypeDef *htim, uint16_t analog_value)
54 {
55     uint32_t compare;
56
57     if (analog_value > ARDUINO_ADC_MAX) {
58         analog_value = ARDUINO_ADC_MAX;
59     }
60     /* Map 0--1023 linearly onto 0--1000 timer counts. */
61     compare = ((uint32_t)analog_value * LED_PWM_COUNTS) / ARDUINO_ADC_MAX;
62     __HAL_TIM_SET_COMPARE(htim, TIM_CHANNEL_1, compare);
63     return compare;
64 }
65
66 uint32_t Task4_SetServoPosition(TIM_HandleTypeDef *htim,
67                                 uint16_t analog_value)
68 {
69     uint32_t pulse;
70
71     if (analog_value > ARDUINO_ADC_MAX) {
72         analog_value = ARDUINO_ADC_MAX;
73     }
74     /* At a 1 MHz timer tick, the compare value is the pulse width in us. */
75     pulse = SERVO_MIN_PULSE_US
76         + ((uint32_t)analog_value * SERVO_PULSE_RANGE_US) / ARDUINO_ADC_MAX;
77     __HAL_TIM_SET_COMPARE(htim, TIM_CHANNEL_1, pulse);
78     return pulse;
79 }

```

Figure 3 shows both transfer functions. The LED duty cycle and servo pulse width increase monotonically across the complete 10-bit input range.

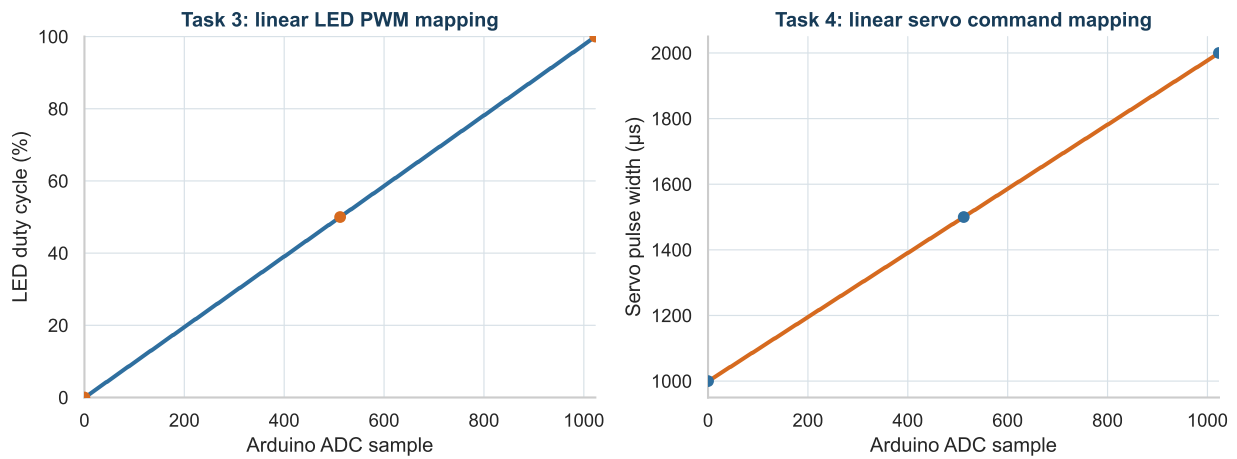


Figure 3: ADC-to-PWM mappings used for LED brightness and servo position.

6 Conclusion

The laboratory integrated three serial interfaces with two timer-controlled outputs. Interrupt-driven UART reception supported a menu-based calculator and parameter entry. I2C transactions selected the Arduino LED interval and retrieved either a reconstructed analog value or the four-byte text response. SPI register transactions acquired the same 10-bit sample as two bytes. Linear timer mappings then converted this sample into a 1 kHz LED duty cycle and a 50 Hz servo pulse. The modular implementation separates terminal control, protocol exchange, and PWM conversion while retaining a single application flow for all four tasks.